DCF 估值分析智能代理

基于大语言模型的贴现现金流自动估值系统

项目技术报告

LLM 模型：	MiniMax-M2.7-highspeed
后端框架：	Python FastAPI
前端框架：	React 18 + TypeScript + Vite
日期：	2026 年 3 月

目录

1 项目概述 **3**
1.1 背景与动机 3
1.2 核心目标 3
1.3 技术选型概览 4

2 系统架构设计 **4**
2.1 整体架构 4
2.2 后端架构 5
2.3 前端架构 5
2.4 API 接口设计 6

3 核心算法与模型 **6**
3.1 DCF 估值理论基础 6
3.2 FCFF 计算 6
3.3 WACC 计算 7
3.4 终端价值 7
3.5 企业价值与每股价值 7
3.6 敏感性分析 8

4 LLM 智能提取模块 **8**
4.1 模型选型 8
4.2 PDF 智能提取策略 8
4.3 结构化提取 Prompt 设计 9
4.4 推理标签处理 9

5 前端交互设计 **9**
5.1 页面结构 9
5.2 主要交互功能 10
5.3 可视化图表 10

6 测试验证 **10**
6.1 测试样本 10
6.2 数据提取结果 10
6.3 DCF 估值计算结果 11
6.4 敏感性分析矩阵 11
6.5 AI 估值叙述 11
6.6 端到端性能 12

7 技术挑战与解决方案 **12**

7.1 挑战一：openai/httpx 版本不兼容 12
7.2 挑战二：LLM 推理标签导致 JSON 解析失败 12
7.3 挑战三：PDF 关键数据分散 12
7.4 挑战四：环境变量加载路径 13

8 总结与展望 13
8.1 项目成果 13
8.2 未来改进方向 13

A 核心数据模型定义 13

1 项目概述

1.1 背景与动机

随着金融市场数据量的持续激增和财务分析复杂度的不断提升，传统的 DCF（Discounted Cash Flow，贴现现金流）估值方法面临显著的效率瓶颈。传统方法依赖大量人工操作：分析师需要手动从上市公司年报中提取关键财务数据、逐项整理 Excel 模型、反复调整假设参数，整个过程耗时长且容易出错。

本项目旨在设计并实现一个**基于大语言模型与自动化工作流的 DCF 估值智能代理**，实现从企业财报上传到最终估值结果的全自动化流程，为投资分析师和金融研究人员提供高效、准确的估值支持工具。

1.2 核心目标

1. **自动化数据提取**：上传 PDF 格式的上市公司年报，由 AI 自动提取 20+ 项关键财务指标。
2. **完整 DCF 建模**：基于 FCFF 方法构建自由现金流折现模型，计算企业价值与每股内在价值。
3. **多维敏感性分析**：生成 WACC 与永续增长率的交叉敏感性矩阵，量化参数假设对估值的影响。
4. **AI 估值报告**：利用大语言模型自动生成专业级估值分析叙述。
5. **美观的交互界面**：提供完整的前后端系统，支持中英文双语切换与深色/浅色主题。

1.3 技术选型概览

表 1: 系统技术栈

层级	技术	说明
后端框架	Python FastAPI + Uvicorn	高性能异步 Web 框架
数据模型	Pydantic v2	类型安全的请求/响应校验
PDF 解析	pdfplumber	精确提取 PDF 文本与表格
LLM 服务	MiniMax-M2.7-highspeed	推理型大语言模型
LLM SDK	OpenAI Python SDK	兼容 MiniMax API 端点
前端框架	React 18 + TypeScript	组件化前端开发
构建工具	Vite 5	极速热更新开发体验
UI 组件库	Ant Design 5	企业级 React 组件
样式方案	TailwindCSS	原子化 CSS 实用类
图表库	ECharts	专业级交互式图表
状态管理	Zustand	轻量级全局状态
国际化	react-i18next	中英文双语切换

2 系统架构设计

2.1 整体架构

系统采用经典的**前后端分离**架构，前端通过 Vite 开发服务器代理将 API 请求转发至后端 FastAPI 服务。整体数据流如下：

系统数据流

1. 用户上传 PDF 财务报告或手动输入财务数据；
2. 后端使用 pdfplumber 智能提取 PDF 关键页面文本；
3. 提取的文本发送至 MiniMax LLM，返回结构化 JSON 财务数据；
4. 用户可在界面中审查和编辑提取结果，并调整 DCF 参数；
5. 后端 DCF 引擎执行估值计算、敏感性分析；
6. LLM 生成专业估值叙述报告；
7. 前端以图表和仪表盘形式展示完整估值结果。

2.2 后端架构

后端采用**分层架构**，清晰划分 API 层、服务层和模型层：

表 2: 后端模块划分

模块	文件	职责
API 层	`api/upload.py`	PDF 上传与数据提取端点
	`api/analysis.py`	DCF 计算与敏感性分析端点
	`api/report.py`	报告导出端点（预留）
服务层	`services/pdf_service.py`	PDF 智能文本提取
	`services/llm_service.py`	MiniMax LLM 封装
	`services/extractor_service.py`	LLM 财务数据提取
	`services/dcf_service.py`	DCF 计算引擎
模型层	`models/schemas.py`	Pydantic 数据模型定义
配置层	`config.py`	环境变量与配置加载
	`main.py`	FastAPI 应用入口

2.3 前端架构

前端基于 React 单页应用（SPA），使用 React Router 实现页面路由，Zustand 管理全局状态：

表 3: 前端页面与组件

类型	组件	功能
页面	`HomePage`	首页与引导入口
	`AnalysisPage`	数据输入与参数调整
	`ResultPage`	估值结果仪表盘
组件	`FileUpload`	PDF 拖拽上传
	`ManualInput`	手动数据输入表单
	`ParamPanel`	DCF 参数滑块面板
	`CashFlowChart`	FCF 预测柱状图
	`SensitivityTable`	敏感性分析热力图
	`WaterfallChart`	估值瀑布图

2.4 API 接口设计

表 4: RESTful API 接口一览

方法	路径	说明
GET	`/api/health`	健康检查
POST	`/api/extract/upload`	上传 PDF，提取文本并调用 LLM 返回结构化财务数据
POST	`/api/extract/text`	从纯文本中调用 LLM 提取财务数据
POST	`/api/calculate`	执行 DCF 估值计算，返回完整估值结果
POST	`/api/sensitivity`	生成 WACC-增长率敏感性分析矩阵
POST	`/api/narrative`	调用 LLM 生成专业估值叙述报告

3 核心算法与模型

3.1 DCF 估值理论基础

DCF（Discounted Cash Flow）估值法的核心思想是：**一项资产的价值等于其未来产生的所有自由现金流的折现值之和**。本系统采用 FCFF（Free Cash Flow to Firm，企业自由现金流）方法进行估值。

3.2 FCFF 计算

企业自由现金流的计算公式为：

$$\text{FCFF} = \text{NOPAT} + \text{D\&A} - \text{CapEx} - \Delta\text{NWC} \tag{1}$$

其中：

- $\text{NOPAT} = \text{EBIT} \times (1 - T_c)$，即税后净营业利润；
- D&A 为折旧与摊销；
- CapEx 为资本性支出；
- ΔNWC 为营运资本变动。

在预测期内，系统基于用户设定的营收增长率、经营利润率等参数逐年投射 FCFF：

$$\text{Revenue}_t = \text{Revenue}_{t-1} \times (1 + g_{\text{rev}}) \tag{2}$$

$$\mathrm{FCFF}_t = \mathrm{Revenue}_t \times m_{\mathrm{op}} \times (1-T_c) + \mathrm{Revenue}_t \times r_{\mathrm{DA}} - \mathrm{Revenue}_t \times r_{\mathrm{CapEx}} - \mathrm{Revenue}_t \times r_{\mathrm{NWC}} \tag{3}$$

3.3 WACC 计算

加权平均资本成本（WACC）是企业融资成本的加权平均，用作折现率：

$$\mathrm{WACC} = \frac{E}{E+D} \times R_e + \frac{D}{E+D} \times R_d \times (1-T_c) \tag{4}$$

其中权益成本 R_e 通过 CAPM（资本资产定价模型）计算：

$$R_e = R_f + \beta \times (R_m - R_f) \tag{5}$$

- R_f：无风险利率（默认取中国十年期国债收益率 2.5%）
- β：股票贝塔系数
- R_m：市场预期回报率（默认 9%）
- R_d：债务成本
- $E/(E+D)$：权益占总资本比重
- $D/(E+D)$：债务占总资本比重

3.4 终端价值

预测期结束后，采用 Gordon 永续增长模型计算终端价值（Terminal Value）：

$$\mathrm{TV} = \frac{\mathrm{FCFF}_n \times (1+g_\infty)}{\mathrm{WACC} - g_\infty} \tag{6}$$

其中 g_∞ 为永续增长率（默认 2.5%），约束条件为 $\mathrm{WACC} > g_\infty$。

3.5 企业价值与每股价值

$$\mathrm{EV} = \sum_{t=1}^{n} \frac{\mathrm{FCFF}_t}{(1+\mathrm{WACC})^t} + \frac{\mathrm{TV}}{(1+\mathrm{WACC})^n} \tag{7}$$

$$\text{Equity Value} = \mathrm{EV} - \text{Total Debt} + \text{Cash} \tag{8}$$

$$\text{Per Share Value} = \frac{\text{Equity Value}}{\text{Shares Outstanding}} \tag{9}$$

3.6 敏感性分析

系统构建以 WACC 和永续增长率 g_∞ 为双轴的敏感性矩阵。对于基准值附近 $\pm 2\%$ 范围内的 9 个 WACC 值和 7 个增长率值，分别计算每股价值，生成 9×7 的矩阵。该矩阵以热力图形式可视化，帮助投资者直观理解关键假设对估值的影响程度。

4 LLM 智能提取模块

4.1 模型选型

本系统选用 **MiniMax-M2.7-highspeed** 作为大语言模型，该模型具有以下特点：

- 支持 OpenAI 兼容 API，可直接使用 OpenAI Python SDK 调用；
- 内置推理能力（Chain-of-Thought），适合复杂财务数据解析；
- 高速推理，满足实时交互需求；
- 原生支持中英文，适合中国 A 股上市公司年报处理。

通过 OpenAI SDK 的 `base_url` 参数连接 MiniMax API：

Listing 1: MiniMax LLM 客户端初始化

```python
from openai import AsyncOpenAI

client = AsyncOpenAI(
    api_key=MINIMAX_API_KEY,
    base_url="https://api.minimaxi.com/v1",
)
```

4.2 PDF 智能提取策略

面对 200+ 页的中国上市公司年报，直接将全文发送给 LLM 既不经济也超出上下文限制。系统采用**关键词权重评分**策略：

1. 遍历 PDF 每一页，提取文本内容；
2. 对每页文本计算关键词匹配得分（关键词包括 “营业收入”、“净利润”、“资产负债表”、“现金流” 等 17 个中英文财务术语）；
3. 按得分从高到低排序，优先选取高相关性页面；
4. 累计字符数不超过 30,000 字（约涵盖 15–20 页高价值内容）；
5. 按原始页码重新排序后拼接，送入 LLM 进行提取。

4.3 结构化提取 Prompt 设计

提取 Prompt 使用中文，精确定义 21 个目标字段及其含义，并明确约定：

- 所有金额统一为“元”（自动处理万元/亿元转换）；
- 所有比率使用小数（如 14.23% → 0.1423）；
- 缺失值基于其他已知数据合理估算；
- 仅返回 JSON 对象，不包含 markdown 标记。

4.4 推理标签处理

MiniMax-M2.7 内置推理模式，会在响应中输出 `<think>...</think>` 思考过程。系统通过正则表达式自动剥离推理标签，再从剩余文本中定位并解析 JSON 对象：

Listing 2: 推理标签处理

```python
import re, json

def _extract_json(raw: str) -> dict:
    text = raw.strip()
    text = re.sub(r"<think>.*?</think>", "", text,
                  flags=re.DOTALL).strip()
    match = re.search(r"\{[\s\S]*\}", text)
    if match:
        return json.loads(match.group())
    return json.loads(text)
```

5 前端交互设计

5.1 页面结构

前端共包含三个主要页面，遵循**向导式**交互流程：

首页（HomePage） 项目介绍、功能亮点展示与“开始分析”入口按钮。

分析页（AnalysisPage） 左侧提供“上传报告”与“手动输入”两个标签页，右侧为 DCF 参数实时调整面板（含 WACC、增长率、利润率等滑块），底部为“计算估值”按钮。

结果页（ResultPage） 展示估值概要指标、FCF 预测柱状图、敏感性分析热力图、估值瀑布图以及 AI 生成的估值叙述。

5.2 主要交互功能

- **PDF 拖拽上传**：支持点击或拖拽上传 PDF 文件，自动触发数据提取。
- **实时数据编辑**：AI 提取的财务数据以可编辑表单展示，用户可逐项修正。
- **参数滑块调节**：WACC、营收增长率、经营利润率等参数通过滑块和输入框实时调整。
- **一键计算**：点击按钮依次执行 DCF 计算、敏感性分析、AI 叙述生成，完成后自动跳转至结果页。
- **双语切换**：导航栏提供中文/英文一键切换，所有界面文本实时更新。
- **主题切换**：支持浅色/深色模式切换，适应不同工作环境。

5.3 可视化图表

系统使用 ECharts 库实现三类专业级金融图表：

1. **FCF 预测柱状图**：展示 5 年预测期内的自由现金流趋势。
2. **敏感性热力图**：以 WACC 为纵轴、永续增长率为横轴的 9×7 矩阵，颜色深浅表示每股价值大小。
3. **估值瀑布图**：从企业价值出发，经过减除债务、加回现金，最终得到每股价值的桥接图。

6 测试验证

6.1 测试样本

使用**苏州天华新能源科技股份有限公司（300390）2025 年年度报告**（226 页，约 24.6 万字）作为测试样本，验证系统全流程功能。

6.2 数据提取结果

系统成功从 PDF 中自动提取以下关键财务数据（表 5）：

表 5: LLM 自动提取结果

指标	提取值	单位
公司名称	苏州天华新能源科技股份有限公司	
股票代码	300390	—
报告年度	2025	年
营业收入	7,548,826,104.54	元
营收增长率	14.23	%
净利润	402,189,233.13	元
经营利润率	5.33	%
总债务	2,349,172,637.01	元
货币资金	4,289,026,296.41	元
总股本	830,750,788	股
有效税率	1.42	%
Beta 系数	1.0	—

6.3 DCF 估值计算结果

基于提取数据与默认 DCF 参数（WACC=10%，营收增长率 =14.23%，经营利润率 =5.33%，预测 5 年），系统输出以下估值结果：

表 6: DCF 估值结果

指标	数值
WACC（加权平均资本成本）	10.00%
企业价值（Enterprise Value）	3,929,036,401.44 元
权益价值（Equity Value）	3,939,589,827.81 元
每股价值	4.74 元
预测年数	5 年

6.4 敏感性分析矩阵

敏感性分析生成了 9×7 的每股价值矩阵，WACC 范围 8%–12%，永续增长率范围 1.0%–4.0%，帮助投资者理解估值的参数敏感度。

6.5 AI 估值叙述

系统成功调用 MiniMax LLM 生成约 4,700 字的专业估值分析报告，涵盖公司概况、关键指标、WACC 假设分析、现金流预测、终端价值方法论、估值结论及风险提示等内容。

6.6 端到端性能

表 7: 系统性能指标

环节	耗时
PDF 上传与文本提取	∼5 秒
LLM 数据提取	∼45 秒
DCF 计算	<1 秒
敏感性分析	<1 秒
AI 叙述生成	∼50 秒
全流程总计	∼100 秒

7 技术挑战与解决方案

7.1 挑战一：openai/httpx 版本不兼容

问题： `openai==1.51.0` 在创建 `httpx.AsyncClient` 时传递了 `proxies` 参数，而 `httpx>=0.28` 已移除该参数，导致 `AsyncClient.__init__() got an unexpected keyword argument 'proxies'` 错误。

解决： 将 openai 升级至 `>=1.58.0,<2.0.0`（实际安装 1.109.1），该版本兼容 httpx 0.28+。

7.2 挑战二：LLM 推理标签导致 JSON 解析失败

问题： MiniMax-M2.7-highspeed 是推理型模型，在每次响应前自动输出 `<think>...</think>` 思考过程。当 `max_tokens` 不足时，模型将全部 token 用于推理，最终 JSON 未能输出。

解决：

1. 将 `max_tokens` 从 2,000 提升至 16,000，为思考与输出留出充足空间；
2. 编写正则表达式处理器，自动剥离 `<think>` 标签后提取 JSON 对象。

7.3 挑战三：PDF 关键数据分散

问题： 226 页年报的关键财务数据分散在不同章节（第 7–9 页的主要指标、第 84 页起的资产负债表、利润表等），直接提取前 12,000 字符无法覆盖核心数据。

解决： 实现基于关键词权重评分的智能页面选择算法，优先提取包含 “营业收入”“净利润”“资产负债表” 等 17 个关键词的高价值页面，总提取量控制在 30,000 字以内。

7.4 挑战四：环境变量加载路径

问题：`config.py` 中 `load_dotenv()` 默认在工作目录查找 `.env` 文件，但实际文件位于 `backend/.env` 子目录。

解决：显式指定路径 `load_dotenv(_backend_dir / ".env")`，确保从项目根目录启动时正确加载配置。

8 总结与展望

8.1 项目成果

本项目成功构建了一套完整的 DCF 估值分析智能代理系统，实现了以下成果：

- 实现了从 PDF 年报上传到 AI 自动提取 21 项财务指标的全自动化流程；
- 构建了完整的 FCFF-DCF 估值模型引擎，支持 5 年投射与 Gordon 永续增长终端价值；
- 生成 9×7 的 WACC-增长率敏感性矩阵，以热力图可视化呈现；
- 利用 MiniMax LLM 自动生成 4,000+ 字的专业估值叙述报告；
- 打造了美观的全栈 Web 应用，支持中英文双语与深色/浅色主题。

8.2 未来改进方向

1. **多期数据分析**：支持多年财报对比，自动追踪关键指标趋势。
2. **行业对标**：引入同行业可比公司数据，提供相对估值参考。
3. **实时市场数据**：接入股票行情 API，自动获取当前股价与 Beta 值。
4. **蒙特卡洛模拟**：基于参数分布进行随机模拟，输出估值概率分布。
5. **报告导出**：支持将估值结果导出为 PDF/Excel 格式的专业报告。
6. **多模型支持**：适配更多 LLM（如 GPT-4o、Claude 等），对比不同模型的提取准确度。

A 核心数据模型定义

以下为系统核心 Pydantic 数据模型，定义了前后端之间的数据契约：

Listing 3: FinancialData 模型

```python
class FinancialData(BaseModel):
    company_name: str = ""
    ticker: Optional[str] = None
    currency: str = "CNY"
    fiscal_year: int = 2025
    revenue: float = 0
    revenue_growth: float = 0
    operating_income: float = 0
    operating_margin: float = 0.15
    net_income: float = 0
    depreciation_amortization: float = 0
    capital_expenditure: float = 0
    change_in_working_capital: float = 0
    total_debt: float = 0
    cash_and_equivalents: float = 0
    shares_outstanding: float = 0
    tax_rate: float = 0.25
    beta: float = 1.0
    risk_free_rate: float = 0.03
    market_return: float = 0.09
    cost_of_debt: float = 0.05
    current_stock_price: Optional[float] = None
```

Listing 4: DCFParameters 模型

```python
class DCFParameters(BaseModel):
    projection_years: int = 5
    revenue_growth_rate: float = 0.08
    terminal_growth_rate: float = 0.025
    operating_margin: float = 0.15
    tax_rate: float = 0.25
    capex_ratio: float = 0.05
    da_ratio: float = 0.04
    nwc_ratio: float = 0.02
    wacc: float = 0.1
    discount_rate_override: Optional[float] = None
```

Listing 5: DCFResult 模型

```python
class DCFResult(BaseModel):
    projections: list[FCFProjection]
    terminal_value: float
```

```
    pv_terminal_value: float
    pv_fcf_sum: float
    enterprise_value: float
    equity_value: float
    per_share_value: float
    current_price: Optional[float] = None
    upside_downside: Optional[float] = None
    wacc_used: float
    terminal_growth_used: float
```